

Assignment:- 2

Karnaugh Maps (K-maps)

- A **Karnaugh map** is a graphical method for simplifying Boolean algebra expressions.
- It organizes truth-table values into a grid so adjacent cells differ by **only one variable** (Gray code ordering).
- Goal: **minimize logic** (fewer gates) by grouping 1's (for SOP) or 0's (for POS) into powers of two.

Types of K-maps-

1. 2 variable
2. 3 variable
3. 4 variable

3. Rules / Steps for Using K-maps

1. **Draw the map**
 - Choose 2, 3, or 4 variable grid.
 - Label rows/columns in **Gray code** order so only one bit changes between neighbors.
2. **Fill in the map**
 - For **Sum-of-Products (SOP)**: place 1's where the function outputs 1.
 - For **Product-of-Sums (POS)**: place 0's where the function outputs 0.
3. **Group adjacent cells**
 - Groups must be rectangles with sizes = powers of two: 1, 2, 4, 8...
 - Valid shapes: 1×1, 1×2, 2×1, 2×2, 4×1, etc.
 - Wrap-around edges are adjacent (top–bottom and left–right touch).
4. **Make the largest group possible always while grouping**
 - Each group should cover the maximum number of 1's (or 0's) to eliminate the most variables.
5. **Simplify**
 - For each group, write the product (for SOP) of variables that **do not change** within that group.
 - Combine all group terms with OR (for SOP) or AND (for POS).
6. **Check for coverage**

- Ensure every 1 (or 0) is covered at least once.
- Essential groups (covering unique 1's) must be included.

7. Don't care condition

- In this condition we do not care about the output for the corresponding cell and mark it with an 'X' sign.
- We can include this X in our grouping as per need

Page No.
 Date

$$F(A, B, C, D) = \sum m(0, 1, 5, 6, 7, 11, 12, 13, 15)$$

AB \ CD	00	01	11	10
00	1	1	0	0
01	0	1	1	1
11	1	1	1	0
10	0	0	1	0

$$\bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + BD + ACD + \bar{A}BC$$

$$Y = \bar{A}BC + A\bar{B}C + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C}$$

A \ BC	00	01	11	10
0	1	0	1	0
1	1	1	0	0

$$f = \sum m(0, 1, 3, 4, 7, 9, 11, 13, 15)$$

AB \ CD	00	01	11	10
00	1	1	1	0
01	1	0	1	0
11	0	1	1	0
10	0	1	1	0

$$Y = \bar{A}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C} + AD + CD$$

$$F(A, B, C) = \bar{A}B + A\bar{B} + ABC$$

A \ BC	00	01	11	10
0	0	0	1	1
1	1	1	1	0

Page No.
 Date

$F(A, B, C, D) = \sum m(0, 5, 9, 12, 13, 14, 15) +$
 $d(1, 2, 3, 4)$

AB \ CD	00	01	11	10
00	1	X	X	X
01	X	1		
11	1	1	1	1
10		1		

$Y = \bar{A}\bar{B} + \bar{C}D + AB$

AB \ CD	00	01	11	10
00	0	1	0	0
01	0	1	1	1
11	1	1	1	0
10	0	0	1	0

Quine–McCluskey Method (Tabular Method)

The **Quine–McCluskey method** is a step-by-step, tabular way to simplify Boolean expressions. It is especially useful when the number of variables is too large for a Karnaugh Map.

Why It Is Needed

- Digital circuits use Boolean logic.
- Simplifying expressions reduces the number of logic gates.
- This lowers **cost, size, and delay** of the circuit.

Key Terms

- **Minterm:** An AND term where the function is 1 for a specific input combination.
- **Prime Implicant:** A group of minterms combined together that cannot be reduced further.
- **Essential Prime Implicant:** A prime implicant that covers at least one minterm not covered by any other.

Steps of the Method

1. **List the minterms** where the output is 1.
2. **Group minterms** by the number of 1s in their binary form.
3. **Combine adjacent groups** that differ by exactly one bit → replace the differing bit with a dash (–).
4. **Repeat combining** until no further merging is possible.
 - Terms that cannot be combined further = **Prime Implicants**.
5. **Make a Prime Implicant Chart**: rows = prime implicants, columns = minterms.
6. **Find Essential Prime Implicants**: look for columns with only one “X”.
7. **Cover remaining minterms** by selecting the fewest extra prime implicants.
8. The final set gives the **minimal Sum of Products (SOP)** expression.

Example (Outline)

For

$$F(A,B,C,D) = \Sigma m(0,1,2,5,6,7,8,9,10,14)$$

- Step 1: Convert minterms to binary and group them.
- Step 2–3: Combine until no more reduction is possible.
- Step 4: Identify prime implicants.
- Step 5–7: Select essentials and cover remaining terms.
- Step 8: Write the simplified Boolean function.

Advantages

- Works for **any number of variables**.
- Always gives the **minimum expression**.

Disadvantages

- Becomes lengthy by hand if variables $> 5-6$.
- Better handled by computer programs for large cases.

Applications

- Digital circuit design.
- Logic minimization in compilers and CAD tools.
- Teaching and learning Boolean simplification.